

Assignment-04 Report

[Distributed Programming]

Programmazione Concorrente e Distribuita [25-26]

Alex Mazzoni
alex.mazzoni3@studio.unibo.it

June 10, 2026

Contents

1	Exercise 1: Distributed Smart Home Alarm System (Scala - Pekko Cluster)	3
1.1	Analisi del Problema	3
1.1.1	Disaccoppiamento e Service Discovery	3
1.1.2	Gestione dello Stato e Sicurezza nel Recovery	3
1.1.3	Seed Nodes e Configurazione Dinamica	3
1.2	Design e Implementazione	4
1.2.1	Cluster Roles e Configurazione dei Nodi	4
1.2.2	Service Discovery tramite Receptionist	4
1.2.3	Implementazione del Safe Recovery Mode	4
1.2.4	Serializzazione dei Messaggi	4
1.2.5	Gestione dei Seed Nodes	4
2	Exercise 2: Distributed Tic-Tac-Toe (Java - RMI)	5
2.1	Analisi del Problema	5
2.1.1	Sincronizzazione e Deadlock	5
2.1.2	Disconnessioni e "Stanze Zombie"	5
2.1.3	Limiti di Serializzazione in Rete	5
2.2	Design e Implementazione	6
2.2.1	Architettura a Callback (RPC Bidirezionale)	6
2.2.2	Heartbeat Asincrono e Rilevamento Guasti	6
2.2.3	Sincronizzazione e Risorse Condivise	6
2.2.4	Ciclo di Vita, Registry e Pulizia Risorse	6
3	Exercise 3: Distributed Critical Sections with a Message-Oriented Middle-ware (RabbitMQ)	7
3.1	Analisi del Problema	7
3.1.1	Analisi dell'Algoritmo di Ricart-Agrawala	7

3.1.2	Inizializzazione dinamica della rete	8
3.1.3	Fairness delle richieste	8
3.2	Design e Implementazione	8
3.2.1	Architettura di Rete a Topic	8
3.2.2	Protocollo di Discovery Bidirezionale	8
3.2.3	Orologi Logici e Risoluzione dei Pareggi	9
3.2.4	Sincronizzazione Locale dei Thread	9

1 Exercise 1: Distributed Smart Home Alarm System (Scala - Pekko Cluster)

L'obiettivo dell'esercizio è l'estensione del sistema di allarme domestico precedentemente sviluppato, evolvendolo da un sistema locale a scambio di messaggi ad attori, ad un'architettura completamente distribuita basata su Apache Pekko Cluster. Il sistema mantiene intatti gli stati principali, ma il modo in cui i diversi attori vengono creati, è differente, in questa implementazione vengono istanziati su nodi differenti.

1.1 Analisi del Problema

Nel passaggio da un modello locale a uno distribuito, l'astrazione degli attori si è rivelata fondamentale, ma ha introdotto nuove sfide architetturali legate alla topologia della rete, alla scoperta dei servizi e alla tolleranza ai fallimenti dei singoli nodi.

1.1.1 Disaccoppiamento e Service Discovery

In un'architettura distribuita, la gerarchia rigida padre-figlio utilizzata per generare i sensori non è più applicabile, poiché la creazione diretta tramite `context.spawn` vincola gli attori alla medesima JVM (stesso nodo locale). È emersa quindi la necessità teorica di un meccanismo di *Service Discovery* che permetta ai vari nodi di registrarsi e rintracciarsi dinamicamente all'interno della rete.

1.1.2 Gestione dello Stato e Sicurezza nel Recovery

Un nodo all'interno di un cluster può subire crash, partizioni di rete o essere riavviato intenzionalmente. Quando questo accade lo stato di memoria a stati del sistema di allarme principale (il Guardian), viene irrimediabilmente perso. Il problema richiede pertanto la progettazione di una fase di *Safe Recovery*, uno stato di isolamento fiduciario in cui il sistema sospende il suo funzionamento automatico, e richiede il PIN da parte dell'utente prima di riprendere il normale funzionamento.

1.1.3 Seed Nodes e Configurazione Dinamica

In un cluster Pekko, i nodi devono conoscere almeno un *Seed Node* per poter effettuare il bootstrap e unirsi alla rete. Configurare un singolo nodo come seed introduce un *Single Point of Failure* architetturale: se tale nodo dovesse fallire e venire riavviato, al suo risveglio non troverebbe altri seed a cui appoggiarsi, finendo per fondare un nuovo cluster isolato indipendente da quello superstite.

1.2 Design e Implementazione

Per affrontare queste sfide, è stata riprogettata l'architettura rimuovendo la dipendenza dal **SensorsManager** e **SensorsGroup**, sfruttando pienamente le primitive di routing e discovery di Pekko Cluster, i sensori vengono creati in modo distribuito.

1.2.1 Cluster Roles e Configurazione dei Nodi

Il Main `App.scala` è stato strutturato in modo da poter avviare nodi differenti basati sulla proprietà **AlarmSystemRoles**. Quando un nodo viene istanziato, assume un ruolo specifico **Guardian**, **Keypad**, **Siren**, **Sensor**, ovvero gli attori possibili.

1.2.2 Service Discovery tramite Receptionist

Il pattern utilizzato è quello di **Publish / Subscribe** grazie all'attore di sistema **Receptionist**. Ogni entità passiva registra la propria **ServiceKey** al momento del setup. Contemporaneamente, gli attori che necessitano di inviare messaggi si iscrivono agli aggiornamenti di tali chiavi tramite un `messageAdapter`.

Nel **KeypadActor**, questo meccanismo è stato sfruttato per implementare un comportamento reattivo, esso è inizializzato in uno stato di `waitingForGuardian` e transita allo stato `ready` solo quando riceve una **Receptionist.Listing** che attesta la presenza di almeno un Guardian, evitando di inviare PIN a vuoto.

1.2.3 Implementazione del Safe Recovery Mode

Per soddisfare i requisiti di tolleranza ai guasti, la macchina a stati del **AlarmSystem-Guardian** è stata dotata di un nuovo stato iniziale `recovery`. Al momento dell'avvio (o riavvio), l'attore entra in questo stato sicuro. Solo alla ricezione del comando `VerifyPin` contenente la password corretta, il Guardian transita nello stato operativo `disarmed`.

1.2.4 Serializzazione dei Messaggi

Poiché i messaggi ora viaggiano su rete TCP/UDP tra nodi clusterizzati, abbiamo marcato tutti i messaggi (**Command**) e gli eventi di dominio tramite il trait `CborSerializable`. Questo indica a Pekko di utilizzare la serializzazione Jackson CBOR.

1.2.5 Gestione dei Seed Nodes

A livello infrastrutturale, la tolleranza ai fallimenti improvvisi è garantita dall'integrazione dello *Split-Brain Resolver (SBR)* fornito nativamente da Pekko. Configurando la strategia di `keep-majority`. Per permettere ai nodi di mantenere lo stesso cluster, la configurazione condivisa (in `application.conf`) è stata strutturata per definire un elenco di seed nodes multipli (porte 2551 e 2552). In questo modo, il crash di un singolo nodo fondatore non distrugge la membership del cluster, e al suo riavvio esso utilizzerà il seed secondario superstite per ricongiungersi correttamente alla rete esistente.

2 Exercise 2: Distributed Tic-Tac-Toe (Java - RMI)

L'obiettivo dell'esercizio è la progettazione e implementazione del gioco Tic-Tac-Toe multiplayer. L'architettura del sistema si fonda sui paradigmi della programmazione concorrente e degli oggetti distribuiti. Per far interagire i giocatori attraverso la rete, è stato impiegato Java RMI come meccanismo RPC di base.

2.1 Analisi del Problema

Inizialmente sono state analizzate le criticità legate alla gestione di un server che deve far comunicare più client in tempo reale. I problemi principali si sono concentrati sulla concorrenza, sulla gestione della rete e sulla stabilità del sistema a lungo termine.

2.1.1 Sincronizzazione e Deadlock

Poiché il server RMI gestisce ogni richiesta in arrivo su un thread separato, più giocatori possono tentare di accedere allo stato della partita contemporaneamente. Questo richiede l'uso della mutua esclusione per proteggere la logica del gioco. Tuttavia, bloccare l'intero oggetto server durante una chiamata di rete verso un client introduceva un gravissimo rischio di deadlock.

2.1.2 Disconnessioni e "Stanze Zombie"

In un sistema distribuito basato su RMI, il server è intrinsecamente passivo: non si accorge se un client viene chiuso brutalmente finché non tenta di inviargli un messaggio. Questo rischiava di lasciare le Lobby occupate all'infinito da giocatori "fantasma", causando errori di tipo `GameFullException` per i nuovi giocatori ed esaurendo la memoria del server. Per questo motivo, si è reso necessario progettare un meccanismo proattivo per consentire al server di rilevare i guasti silenziosi e notificare tempestivamente i client rimasti bloccati in attesa del proprio turno.

2.1.3 Limiti di Serializzazione in Rete

RMI richiede che tutti i dati scambiati tra client e server vengano serializzati. Durante l'analisi, è emerso che alcune strutture dati moderne di Java, come `Optional<Player>` utilizzata per indicare il vincitore o il pareggio, non implementano l'interfaccia `Serializable` nativamente, causando il crash delle comunicazioni di rete con una `NotSerializableException`.

2.2 Design e Implementazione

La soluzione adottata separa rigorosamente la logica di gioco dalle comunicazioni di rete, promuovendo l'incapsulamento e garantendo un sistema reattivo e tollerante ai guasti.

2.2.1 Architettura a Callback (RPC Bidirezionale)

Il client esporta la propria interfaccia **RemoteClientListener** passandone il riferimento al server. In questo modo, il server può invocare attivamente i metodi del client per fornirgli aggiornamenti in tempo reale. Per garantire che questi messaggi di rete asincroni non bloccassero o corrompessero l'interfaccia grafica, ogni aggiornamento ricevuto dal listener viene delegato all'EDT utilizzando **SwingUtilities.invokeLater()**.

2.2.2 Heartbeat Asincrono e Rilevamento Guasti

Per ovviare alla natura passiva di RMI, è stato implementato il pattern *Heartbeat*. All'interno di **RemoteGameImpl**, un **Timer** in background viene avviato non appena la lobby viene creata. A intervalli regolari, il timer invoca un metodo sui client connessi, che funge da ping.

Se l'invocazione RPC fallisce lanciando una **RemoteException**, il server deduce istantaneamente che il processo client è stato terminato in modo anomalo. A quel punto, il sistema notifica eventuali client superstiti, e invoca **closeGame()** per smantellare la lobby.

2.2.3 Sincronizzazione e Risorse Condivise

Per risolvere il problema dei deadlock menzionati in analisi, è stato applicato il pattern delle *Open Calls*. Nei metodi del server (**RemoteGame** e **RemoteLobby**), l'accesso alle risorse condivise tra i vari client avviene all'interno di un blocco **synchronized(this)**. Tuttavia, prima di invocare i metodi di rete per notificare l'avversario o effettuare il ping, il blocco sincronizzato viene chiuso. Questo garantisce la thread-safety della logica del gioco, e mantenendo al contempo libere le chiamate di rete da attese bloccanti.

2.2.4 Ciclo di Vita, Registry e Pulizia Risorse

Il sistema gestisce autonomamente le proprie risorse di rete. L'avvio del registro RMI (**rmiregistry**) è stato integrato direttamente nel codice del server. Inoltre, per prevenire l'accumulo di stanze concluse, è stato implementato un meccanismo di auto-pulizia. Come accennato nel precedente paragrafo, sono state gestite granparte delle eccezioni di disconnessioni improvvise, soprattutto per liberare memoria dal server. Nella distruzione di **closeGame()**, il **removeLobby(lobbyString, RemoteGame)** agisce rimuovendo se stessa dalla mappa della Lobby ed esegue **UnicastRemoteObject.unexportObject**, disconnettendosi definitivamente dalla rete.

3 Exercise 3: Distributed Critical Sections with a Message-Oriented Middleware (RabbitMQ)

L'obiettivo dell'esercizio è la progettazione e implementazione di un middleware per la mutua esclusione distribuita, ho deciso di implementarlo con l'algoritmo di **Ricart-Agrawala**. L'architettura del sistema si fonda sui paradigmi della programmazione concorrente e distribuita, abbandonando l'approccio centralizzato in favore di una topologia **peer-to-peer**. Per far interagire i nodi attraverso la rete, è stato impiegato il message broker **RabbitMQ** come infrastruttura di comunicazione.

3.1 Analisi del Problema

Inizialmente è stata analizzata la tecnologia **RabbitMQ** e i suoi modelli di comunicazione, con particolare attenzione al paradigma **publish/subscribe**, con **routing**. In un sistema basato su code di messaggi di tipo publish/subscribe, il broker non mantiene uno storico dei messaggi distribuiti. Se un nodo si connetteva in ritardo, perdeva i messaggi di presentazione inviati precedentemente dagli altri partecipanti. Questo portava ad un problema di inconsistenza, sfasando il calcolo del numero totale di partecipanti attivi N , fondamentale per l'algoritmo al fine di attendere il corretto numero di permessi.

3.1.1 Analisi dell'Algoritmo di Ricart-Agrawala

L'analisi delle criticità si è concentrata sulle fondamenta teoriche dell'algoritmo di **Ricart-Agrawala**, e sull'adattamento all'interno di un'infrastruttura di rete reale e puramente asincrona. L'algoritmo impone che un nodo, per poter accedere alla Sezione Critica, debba inviare una richiesta in broadcast e ottenere il consenso esplicito **OK** da tutti gli altri $N - 1$ partecipanti.

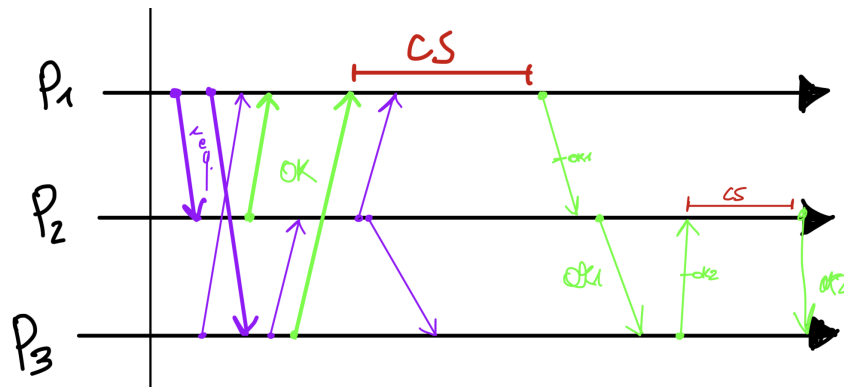


Figure 1: Schema di funzionamento dell'algoritmo di Ricart-Agrawala

3.1.2 Inizializzazione dinamica della rete

Il problema fondamentale nell'implementazione dell'algoritmo è stato quello di definire il numero di partecipanti attivi N , in un sistema distribuito **senza memoria condivisa**. Poiché i nodi possono connettersi e disconnettersi in qualsiasi momento, è stato necessario progettare un meccanismo di discovery dinamico. La soluzione adottata prevede che ogni nodo, al momento della connessione, invii un messaggio di presentazione **HELLO** in broadcast.

3.1.3 Fairness delle richieste

L'algoritmo di **Ricart-Agrawala** basa la propria garanzia di **fairness** sulla capacità di stabilire un ordinamento totale delle richieste. Poiché nei sistemi distribuiti non è possibile avere un clock condiviso, è stato necessario analizzare la teoria delle **happened-before**, tramite l'uso di **Orologi Logici**. La criticità principale sorge in caso di richieste di accesso simultanee, se due o più nodi richiedono l'accesso alla sezione critica nello stesso istante, i loro clock logici risulteranno identici, invalidando la regola di precedenza standard. Era pertanto indispensabile progettare un meccanismo deterministico per i pareggi, per evitare violazioni della mutua esclusione e impedire che il sistema collassasse in un deadlock in cui i nodi si bloccavano a vicenda.

3.2 Design e Implementazione

La soluzione adottata separa rigorosamente la logica dell'algoritmo distribuito dalla gestione del trasporto dei messaggi, promuovendo un sistema reattivo e tollerante alle anomalie di rete.

3.2.1 Architettura di Rete a Topic

Per la comunicazione è stato utilizzato un unico exchange di tipo **TOPIC**. Ogni client dichiara una propria coda privata e la vincola all'exchange tramite due chiavi di routing: una per il broadcast **agrawala.broadcast** per intercettare le richieste globali, e una privata **agrawala.node.[ID]** per ricevere i messaggi di consenso **OK** da ogni singolo nodo.

3.2.2 Protocollo di Discovery Bidirezionale

Durante l'inizializzazione della rete 3.1.2, per ovviare al problema dei nodi ritardatari, è stato implementato un meccanismo di **ping** bidirezionale. Quando la callback riceve un messaggio di **HELLO** da un nodo sconosciuto, non solo aggiorna dinamicamente la grandezza della rete N , ma risponde con un messaggio di **HELLO** inviando a sua volta la propria presentazione in broadcast. Questo garantisce la convergenza rapida e bidirezionale della topologia, allineando la vista di tutti i partecipanti.

3.2.3 Orologi Logici e Risoluzione dei Pareggi

L'ordinamento totale degli eventi è gestito tramite l'implementazione dei **Logical Clocks**. Durante l'invocazione di `enterCS()`, il middleware incrementa il proprio clock logico locale e lo registra come timestamp ufficiale della richiesta (`requestTimestamp`). In caso di richieste concorrenti, l'algoritmo confronta i relativi timestamp e, in presenza di un'esatta parità, utilizza l'ID del nodo come criterio di *tie-breaking* deterministico, assegnando la precedenza al nodo con ID minore.

Se il nodo locale si trova in stato di richiesta e possiede la priorità rispetto alla **REQUEST** ricevuta, la risposta verso l'altro nodo viene posticipata (*deferred*) e il suo ID, viene memorizzato nel Set `pendingNodes`. L'utilizzo di un Set garantisce l'unicità delle richieste pendenti ed evita la duplicazione dei permessi in uscita. Al contrario, se il nodo esterno ha la priorità (o se il nodo locale non è interessato alla sezione critica), il middleware invia immediatamente un messaggio di **OK**. I nodi temporaneamente differiti all'interno del Set verranno sbloccati tutti insieme inviando loro il relativo **OK** solo al momento dell'invocazione di `exitCS()`.

3.2.4 Sincronizzazione Locale dei Thread

Per proteggere lo stato interno del middleware dalle race condition, è stato fatto un uso attento della mutua esclusione locale. L'accesso alle variabili condivise tra il Main Thread e il Thread asincrono di RabbitMQ avviene all'interno di un blocco `synchronized(this)`. Tuttavia, per prevenire deadlock tra la ricezione dei messaggi e l'attesa del completamento dell'algoritmo distribuito, il blocco sincronizzato viene chiuso prima di mettere in pausa il thread applicativo su un **CountDownLatch**. Questo garantisce la thread-safety della logica, mantenendo al contempo la callback di rete sempre libera di processare nuovi messaggi.